TEMA 2

Para la práctica de Entornos de Desarrollo -> Temas 1, 2 y 3. Dice que puede entrar un pseudocódigo también.

QUÉ ES LA INGENIERÍA DEL SOFTWARE

Aquellas técnicas necesarias para desarrollar un software. A la hora de hacer un desarrollo, hay que tener claro y definir poco a poco qué se necesita, ir viendo qué recursos tiene o necesita contratar, las tecnologías que hay que utilizar... Veremos que hay, normalmente, ocho fases en la etapa de un desarrollo, aunque no tiene por qué ser así en todos los casos.

Arquitectura del software: dónde se va a ejecutar. Lo habitual es **cliente/servidor**. El servidor atiende las peticiones del cliente. Es decir, tenemos alguien que está ofreciendo algo (un servicio web, base de datos....) a la que nos conectamos, hacemos una petición y nos devuelve una respuesta.

OCHO FASES DEL DESARROLLO

- Requisitos.
- Análisis.
- Diseño.
- Desarrollo (o implementación).
- Pruebas.
- Documentación.
- Explotación (o implantación).
- Mantenimiento.

Cabe destacar, como dato importante, que todas las metodologías cumplen estas fases, y lo único que cambia es **cómo y cuándo** se recorren.

1. **Requisitos**: un requisito es lo que queremos que nuestra aplicación haga, es decir, las **funcionalidades**. Los requisitos los sacamos hablando o entrevistando al cliente y entendiendo sus necesidades. Todo lo que se hable con un cliente debe quedar documentado o por escrito, porque muchas veces no saben lo que quieren y complican las cosas. A veces, puede haber más de una parte interesada, aparte del cliente en sí. Un ejemplo: un cliente quiere que hagamos un juego de "¿Quién es quién?". El cliente nos tiene que contar cómo va a funcionar el juego de manera informal. Esta es la fase de requisitos.

2. **Análisis**: en la segunda fase, en el análisis, nos ponemos a asentar lo que el cliente quiere; se haría escribiendo un documento técnico o de funcionalidades necesarias para esa aplicación, así como diagramas UML (Lenguaje de Modelado Unificado) de cómo funciona la aplicación. En esta fase de análisis, destacaríamos todas las preguntas posibles y formas de interactuar, por ejemplo.

3. **Diseño**: entonces, pasaríamos a la fase de diseño, que puede ser tanto lo que vemos (interfaces), como diseño de datos (esquemas, por ejemplo) y los recursos físicos o lógicos (cualquier dependencia externa) necesarios para esta fase. En esta fase vemos cómo se organiza nuestra aplicación en distintas partes. Aquí podemos definir exactamente cómo es el tablero, su tamaño, el color de las tarjetas…

4. **Desarrollo (o implementación)**: esta fase requiere de un tiempo desarrollo que puede variar entre meses o años. Además, se puede diseñar en distintos lenguajes de programación. Unas personas se encargarían de la lógica del juego, por ejemplo, habiendo otras muchas que se dividirían entre los distintos miembros del equipo de desarrollo.

5. **Pruebas**: verificar que lo que hemos hecho funciona bien. Haciendo estas comprobaciones, encontraremos qué fallos hay y qué se ha hecho mal. Hay roles que se dedican exclusivamente a buscar estos errores o, a veces, son los mismos desarrolladores los que lo hacen. Se hacen pruebas a nivel de código: que cierto comportamiento funcione como debe o que podamos prever ciertos escenarios, comprobar que las condiciones de victoria se cumplen, entre otros ejemplos.

6. **Documentación**: no va exactamente en esta posición, sino que se hace desde el primer momento. Se explica por qué se han tomado todas las decisiones a lo largo del desarrollo. En Java se usa el *framework* Javadoc, que genera la documentación desde los comentarios del código.

7. **Explotación o implantación**: cuando hemos terminado de probar el programa en local, entregaríamos la versión final al cliente para que lo probara antes de distribuirlo de forma masiva a los usuarios. Cuando este es el caso, que los usuarios o el cliente pueden probarlo, estamos en fase de producción.

8. **Mantenimiento**: puede haber errores tanto lógicos como externos. Existen distintos tipos de mantenimiento. **Correctivo**: se corrigen los defectos. **Perfectivo**: se mejora la funcionalidad existente. **Evolutivo**: se añaden funcionalidades nuevas. **Adaptativo**: se adapta a nuevos entornos.

METODOLOGÍAS DE DESARROLLO DE SOFTWARE

Una metodología es una forma específica de trabajar o de desarrollar. Vemos que las fases anteriores pueden seguir una metodología concreta, es decir, un enfoque en particular con el que se recorren.

Metodología en cascada: es como se hacía al principio. Trabajamos linealmente, y hasta que no termina una fase, no empieza la siguiente. Lo malo es que es difícil calcular el tiempo necesario para finalizar todo el proyecto, por ello, se estiman tiempos **con un margen de error**. Como solución, se implementó el **modelo iterativo** en su lugar, que permite iterar sucesivamente por fases, incluso llegar hasta el final, pero volviendo hacia atrás cuando algo falla. Fue una mejora considerable con respecto al modelo en cascada.

Modelo en espiral: vamos creando poco a poco lo que quiere el cliente, y conforme vamos desarrollando esas funcionalidades, se amplía el proyecto. Esto nos permite tener resultados en menos tiempo. Es decir, parte de una pequeña funcionalidad y todo va creciendo mientras desarrollamos. Al terminar cada vuelta se repite el ciclo, por eso se llama en **espiral**. Es más ágil que el de cascada o el iterativo porque permite un producto funcional en menor tiempo. Sin embargo, no se obtienen tantas indicaciones y puede faltar dirección.

Metodologías ágiles: cortos periodos de tiempo en el que vamos entregando resultados, pero siempre teniendo en cuenta al cliente, que debe ser partícipe de nuestro producto. Dentro de esta categoría, encontramos la metodología **Kanban**, **Scrum** y **XP** (**eXtreme Programming**). Kanban es una manera de organizar el trabajo a través de **columnas** con *post-it* o tarjetas.

Ejemplo gráfico de Kanban:

PENDIENTE	TO DO	IN PROGRESS	TESTING	BLOCKED	DONE
Repasar Sistemas →	Se mueve a esta columna →	Se mueve a la otra	→ [...] →	[...]	→ Hecho

Hay veces que, si te quedas bloqueado o no se puede seguir avanzando, se puede volver a una columna previa. Esto permite moverse libremente por el tablero y permite ver qué se ha estado haciendo con claridad. Los trabajadores van cogiendo tareas del mismo. Las columnas no tienen por qué ser siempre las mismas.

El **Manifiesto Ágil** de 2001 pone a los individuos e interacciones por encima de la metodología, los procesos y las herramientas. Además, fija estos otros valores:

- Software funcionando sobre documentación extensiva.
- Colaboración con el cliente sobre negociación contractual.
- Respuesta ante el cambio sobre seguir un plan.

Esto no significa que la documentación, los contratos o los planes sobren: si hay que elegir, ganan los de la izquierda.

METODOLOGÍAS ÁGILES – SCRUM

Es un modelo de desarrollo incremental. Nos organizamos en pequeñas carreras o ***sprints***, es decir, **ciclos de trabajo corto con un objetivo**. Aquí, siempre se tiene al cliente presente y la comunicación es muy recurrente. Imaginemos que 1 sprint son 2 semanas, entonces, tendríamos que diseñar qué haríamos en ese periodo de tiempo; por ejemplo, hacer 4 funcionalidades. Sigue la siguiente estructura:

1) ***Product Backlog***: registro del producto, todas las funcionalidades que tenemos que hacer para el producto.
2) ***Sprint Planning*** (reunión): nos reunimos con el cliente o un responsable del cliente/producto. En esta reunión se dice todo lo que vamos a hacer y todas las funcionalidades en mente.
3) ***Sprint Backlog***: asignación de tareas y estimación de los tiempos de desarrollo. En este punto, se integra un tablero Kanban colaborativo.
4) ***Daily Scrum***: reuniones internas donde NO participa el cliente, se comprueba cómo va el desarrollo y si están habiendo problemas o si se necesita ayuda. Aquí se suele supervisar qué se hizo el día anterior, si hay algún bloqueo y qué se va a hacer hoy.
5) ***Increment***: cuando se han conseguido avances en el desarrollo o tenemos algo hecho del proyecto.

6) ***Sprint Review***: revisión del *sprint*. Nos reunimos con el cliente y le explicamos qué tenemos hecho. Si está mal o quiere algún cambio, se mueve al *product backlog* y se trabaja en otro momento. Si está bien, se sigue adelante.

7) ***Sprint Retrospective***: entre el equipo, se habla de qué ha salido bien y qué ha salido mal durante ese *sprint*. No se habla de funcionalidades, sino de falta de comunicación o problemas en el día a día a la hora de trabajar en sí. Tras terminar este último ciclo, se vuelve al *Sprint Planning*, y continúa el ciclo.